

EPAM Technical Interview Preparation Guide

Python Web (Django / Flask / FastAPI)

Intermediate Python Web Engineer (A2 equivalent)

Generated: 2026-07-30

Prepared with Codemie AI for EPAM interview preparation

Purpose of This Guide

This guide helps you prepare for an EPAM technical interview for an intermediate-level Python Web role. It focuses on practical ability: building and evolving APIs, working with ORMs and databases, testing, basic security, and deployment/operations awareness.

How to Use This Guide

- Pick one primary framework you will interview with (Django, Flask, or FastAPI) and go deep there.
- Use “Detailed Topics to Review” as a checklist; write short notes and examples from your experience.
- Practice explaining request/response flow and debugging steps—interviews often evaluate reasoning.
- Prepare 1–2 small coding exercises: building an endpoint, validation, ORM query, and a unit test.

Pro Tip: Structure your answers as: what → why → how → trade-offs → how you validate (tests/logs/metrics).

Role and Level Overview

At this level you are expected to work without direct supervision, take ownership of implementation, follow established engineering practices, and communicate directly with the customer when needed (e.g., clarifying requirements, demos, explaining your work).

- Independently implement features and bug fixes in a Python web service.
- Participate in module/component design and explain applied patterns.
- Follow code standards, contribute in code reviews as a reviewer.
- Write reliable tests and understand basic quality control practices.
- Understand REST best practices and common web security risks (OWASP basics).

What to Expect in the Interview

- Conceptual questions on Python, web fundamentals, REST, and your chosen framework.
- A practical coding task (often API-focused): implement an endpoint, add validation, write tests, or debug an issue.
- Questions on ORMs/migrations and writing correct, efficient queries.
- Basic security and production readiness topics: auth headers, CSRF/XSS/SQLi awareness, config/secrets.

- Light DevOps awareness: Docker usage, CI pipeline basics, reading logs and troubleshooting.

Interviewer Focus: Clarity and correctness first; then maintainability (structure, tests, error handling).

If you get stuck: State assumptions, propose a minimal correct solution, then improve it step-by-step.

Technical Interview Focus Areas

- Python language fundamentals relevant to web services (exceptions, functions, OOP basics).
- Python web development: request/response lifecycle, routing, middleware, validation, serialization.
- REST API design and HTTP: idempotency, versioning, pagination, headers, status codes, SSL/TLS basics.
- ORM and database work: migrations, relations, indexing/constraints, avoiding N+1 queries.
- Testing: unit vs integration; mocks/stubs/patch; flakiness and execution time trade-offs.
- Async basics (async/await, event loop) when using async frameworks/components.
- Security: OWASP basics (XSS, CSRF, SQL injection) and secure credential handling.
- Monitoring/logging and deployment basics: Docker as a user, reading logs, simple deployment patterns.

Detailed Topics to Review

1) Web Framework Core (pick one and master)

- Request/response flow in your framework: routing → middleware → handler/controller → response.
- Middleware concept and typical built-ins (auth, logging, CORS, sessions, CSRF for Django).
- Validation & serialization (e.g., Pydantic in FastAPI; DRF serializers; Marshmallow in Flask ecosystems).
- Error handling strategy: mapping exceptions to HTTP responses; consistent error formats.
- WebSockets basics: when used and how they differ from HTTP request/response.

Common Pitfall: Mixing business logic into controllers/routers. Aim for thin handlers and testable service layers.

2) REST & HTTP (intermediate expectations)

- REST best practices: resource naming, idempotency (GET/PUT/DELETE), safe methods, versioning.
- Pagination approaches: limit/offset vs cursor; trade-offs.
- Headers you should confidently use/explain: Authorization, Content-Type, Accept, Cache-Control, User-Agent, Accept-Language.
- Status codes: 200/201/204, 400/401/403/404/409, 422, 500; when to use which.
- TLS/SSL basics: why HTTPS matters and what it protects.

3) ORM, Migrations, and Querying

- Why ORM: productivity and safety vs drawbacks (hidden queries, performance).
- Migrations: why needed, typical workflow, handling backwards-incompatible changes.
- Relations: one-to-many and many-to-many; eager vs lazy loading; avoiding N+1.
- Indexes and constraints: uniqueness, foreign keys; when indexes help.
- Intermediate ORM usage: bulk operations, raw SQL when needed (and how to keep it safe).

Interviewer Focus: Correctness + performance awareness: can you explain how many queries happen and why?

4) Python language topics often tested in web interviews

- Exceptions: custom exceptions, Exception vs BaseException, try/except/else/finally.
- Functions: type hints, docstrings, LEGB scoping rule; higher-order functions basics.
- OOP: class vs instance attributes, @property, classmethod vs staticmethod.
- Async basics: coroutines/tasks/futures; where async helps (I/O-bound) and where it doesn't.

5) Testing (required at A2)

- Testing pyramid: what belongs in unit vs integration vs end-to-end for an API.
- Mock/stub/patch: when to mock (external services) and when not to (your own code).
- Flaky tests: typical causes (timing, shared state, network) and mitigations.
- What makes a good test: independent, readable, one reason to fail, deterministic.

Common Pitfall: Testing implementation details instead of behavior (e.g., asserting internal calls rather than output/side effects).

6) Security basics (Python Web Security – intermediate)

- Credential handling: do not hardcode; use environment variables and secret managers when available.
- OWASP basics: XSS, CSRF, SQL injection—how they appear and how frameworks mitigate them.
- Auth basics: credentials in headers vs request body; why HTTPS is mandatory.
- Input validation and output escaping as default practices.

7) Logging, monitoring, and deployment awareness

- Logging levels and what to log for APIs (request id/correlation id, key events, errors).
- Reading logs/metrics in tools like ELK/Sentry/NewRelic/Datadog (as a user).
- CI basics: run jobs, read pipeline logs, understand common failures.
- Docker basics: run existing images, minor docker-compose edits; basics of image best practices.

What the Interviewer May Look For

- Ability to implement a working solution and own it end-to-end (including tests).
- Correct and practical understanding of HTTP/REST and framework request handling.
- Comfort with ORMs/migrations and relational modeling.
- Basic security awareness and safe defaults.
- Good engineering hygiene: code readability, code review participation, CI familiarity.

Practical Preparation Tips

1. Build a small API (3–5 endpoints) with your chosen framework: CRUD + pagination + validation + error handling.
2. Add ORM models + migrations and practice writing 3 queries: filtering, join/relationship, aggregation.
3. Write tests: at least 5 unit tests and 3 integration tests using a test database or sqlite in-memory (where appropriate).
4. Practice debugging: add structured logs and reproduce a failing request; trace it end-to-end.

5. Review OWASP Top risks relevant to APIs (XSS/CSRF/SQLi) and map each to a mitigation.

Pro Tip: Be ready to explain how you would deploy/run the service locally (env vars, docker-compose, migrations, seed data).

Common Mistakes to Avoid

- Not knowing how your framework processes requests (middleware, routing, serialization).
- Incorrect status codes or inconsistent error responses.
- Ignoring migrations and data evolution (breaking changes without a plan).
- Overusing mocks and ending up with low-confidence tests.
- Leaking secrets (committing .env files, logging tokens/passwords).

Sample Technical Questions

- Explain the request/response lifecycle in Django/Flask/FastAPI. Where does middleware fit?
- Design an endpoint for listing items with pagination, filtering, and sorting. Which query patterns do you use?
- How do you avoid N+1 queries in your ORM? Give an example.
- What is idempotency? Which HTTP methods should be idempotent and why?
- Describe how SQL injection happens and how you prevent it in Python web apps.
- Write a unit test for a function that calls an external API. What do you mock and why?
- When would you choose async endpoints (async/await) and what are the risks?

Final Preparation Checklist

- I can implement an API endpoint with validation, error handling, and correct status codes.
- I can explain middleware and the request lifecycle in my chosen Python framework.
- I can model relationships and write migrations confidently.
- I can write reliable tests (unit + integration) and explain mocking trade-offs.
- I understand basic OWASP risks and can describe practical mitigations.

- I can run the service via Docker/docker-compose and interpret CI pipeline failures.

Closing Advice

At intermediate level, aim to show dependable execution and thoughtful engineering hygiene. Be explicit about assumptions, prioritize correctness, and demonstrate how you ensure quality with tests, reviews, and safe handling of configuration and secrets.